

Conexão com um banco de dados PostgreSQL

Conexão Nativa (libpq)

- A biblioteca C **libpq**, que é a biblioteca utilizada pelas ferramentas de linha de comando do PostgreSQL (como o `psql`), usa o **protocolo frontend/backend** (também conhecido como **protocolo de comunicação** ou "wire protocol") fornecido pelo PostgreSQL. Este é um protocolo binário sobre TCP/IP (ou soquetes de domínio Unix).
 - <https://www.postgresql.org/docs/current/libpq.html>
 - É a forma como a biblioteca python `psycopg2` (vista em aula) se conecta a um banco PostgreSQL
 - **Fluxo de Conexão:**
 1. A aplicação Python chama `psycopg2.connect(dsn)`.
 2. O `Psycopg2` invoca a `libpq` com parâmetros de conexão (host, porta, usuário, senha, dbname).
 3. A `libpq` estabelece uma conexão de soquete com o servidor PostgreSQL.
 4. A autenticação ocorre (por exemplo, SCRAM-SHA-256, MD5 ou TLS).
 5. Consultas SQL/resultados são transmitidos via protocolo de comunicação do PostgreSQL.

Formas de se conectar a um banco de dados PostgreSQL

Existem várias maneiras de se conectar a um banco de dados PostgreSQL, como:

1. Drivers Nativos

- **Mecanismo:** Implementação direta do protocolo de comunicação do PostgreSQL.
- **Exemplos:**
 - **Python:** `psycopg2`, `psycopg3`, `asyncpg`.
 - **Java:** Driver JDBC do PostgreSQL (PGJDBC).
 - **Node.js:** `node-postgres (pg)`.
 - **Go:** `pgx`, `pq`.
 - **C/C++:** `libpq` (API C oficial).
- **Prós:** Eficiente, baixa latência, suporte completo a recursos.
- **Contras:** Específico da linguagem.

2. Drivers Específicos de Linguagem:

- **Python:** `psycopg2`, `psycopg3`, `asyncpg`, `SQLAlchemy` (que pode usar um desses drivers internamente).
- **Java:** Driver JDBC (por exemplo, o driver JDBC oficial do PostgreSQL).
- **Node.js:** `pg (node-postgres)` ou `pg-native` (que usa `libpq`).
- **Ruby:** Gema `pg` (que usa `libpq`).
- **PHP:** Extensão `pgsql` (usa `libpq`) ou `PDO_PGSQL`.
- **Go:** Drivers `pq` ou `pgx`.
- **C#/.NET:** `Npgsql` (o provedor de dados .NET para PostgreSQL).

3. ODBC (Open Database Connectivity)

- **Como funciona:** A aplicação usa a API ODBC. O gerenciador de drivers ODBC carrega o driver ODBC do PostgreSQL (`psqlODBC`). O driver então traduz as chamadas ODBC para o protocolo PostgreSQL (usando `libpq` internamente). A string de conexão é configurada em um Nome de Fonte de Dados (DSN) ODBC ou fornecida diretamente.
- **Mecanismo:** API genérica usando drivers ODBC como uma camada de tradução.
- **Configuração:**
 1. Instale o Driver ODBC do PostgreSQL (por exemplo, `psqlODBC`).
 2. Configure um DSN (Nome de Fonte de Dados).
 3. As aplicações usam a API ODBC (por exemplo, `pyodbc` do Python).
- **Casos de Uso:** Aplicações Windows (Excel, .NET), sistemas legados.
- **Prós:** Multiplataforma, independente de linguagem.
- **Contras:** Sobrecarga, mais lento que drivers nativos.

4. JDBC (Java Database Connectivity)

- **Como funciona:** Uma aplicação Java usa a API JDBC. O driver JDBC do PostgreSQL (uma implementação pura em Java) traduz as chamadas JDBC para o protocolo de comunicação do PostgreSQL. O driver pode se conectar via TCP/IP.
- **Mecanismo:** API padrão Java usando o driver JDBC do PostgreSQL.
- **Configuração:**
 - Adicione postgresql-<version>.jar ao classpath.
 - Conecte via:

```
Connection conn = DriverManager.getConnection(
    "jdbc:postgresql://host:5432/db", "user", "password"
);
```
- **Casos de Uso:** Aplicações Java (Spring, Hibernate).
- **Prós:** Portável entre bancos de dados (com ajustes SQL).
- **Contras:** Apenas Java, leve sobrecarga.

5. Ferramentas de Linha de Comando

- **psql:**
 - Usa libpq para conexões.
 - Sintaxe: psql -h host -p port -U user -d dbname
- **pgcli:** psql aprimorado com autocompletar.

6. Clientes Gráficos

- **Ferramentas:** pgAdmin, DBeaver, TablePlus, DataGrip.
- **Protocolo Subjacente:** A maioria usa libpq ou JDBC internamente.

7. Gateways REST/HTTP

- **Como funciona:** Um middleware como o PostgREST fica em frente ao banco de dados PostgreSQL e traduz requisições HTTP RESTful em operações SQL. O cliente usa HTTP (com JSON) para interagir com o banco de dados.
- **Prós:** Fácil de usar a partir de clientes web, não requer conexão direta com o banco de dados (amigável a firewalls).
- **Contras:** Limitado às capacidades expostas pelo middleware, latência adicional.
- **Mecanismo:** Middleware expondo PostgreSQL via HTTP.
- **Exemplos de Ferramentas:**
 - PostgREST: API REST direta para PostgreSQL.
 - Hasura: Camada GraphQL/HTTP.
- **Casos de Uso:** Aplicativos web/mobile que evitam acesso direto ao banco de dados.

8. Outros Protocolos

- **Foreign Data Wrappers (FDW)**
 - **Mecanismo:** Consultar servidores PostgreSQL remotos como tabelas locais.
 - **Exemplo:**

```
CREATE SERVER remote_db FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host '10.0.0.1', dbname 'target_db', port '5432');
```
- **Protocolos de Replicação e Backup**
 - Ferramentas como pg_basebackup, pg_receivewal ou ferramentas de replicação lógica usam o protocolo de replicação em fluxo do PostgreSQL (uma variante do protocolo de comunicação) para se conectar.
 - **Replicação por Streaming:** Replicação física sobre TCP.
 - **Decodificação Lógica:** Plugins de saída (por exemplo, pgoutput, wal2json) para captura de dados de alteração.
- **dblink:** Uma extensão do PostgreSQL que permite conexões com outros bancos de dados PostgreSQL a partir de uma sessão.

9. Conectores de Pool de Conexões

- O cliente se conecta ao pool (usando um dos métodos acima), e o pool gerencia um conjunto de conexões com o servidor real. Não é um protocolo distinto, mas um retransmissor.
- **Ferramentas:** PgBouncer, Pgpool-II.
- **Função:** Proxy de conexões para reduzir sobrecarga (não é um protocolo de cliente direto).

Considerações-Chave

- **Desempenho:** Drivers nativos (baseados em libpq) > ODBC/JDBC > HTTP.
- **Segurança:**
 - Criptografia TLS/SSL suportada nativamente em todos os drivers.
 - Autenticação: SCRAM-SHA-256, GSSAPI, LDAP, etc.
- **Exemplos de Utilização Adequada de Formas de Conexão:**
 - **Drivers Nativos:** Aplicações de uso crítico (melhor desempenho).
 - **ODBC:** Integração com Windows/legado.
 - **JDBC:** Ecossistemas Java.
 - **HTTP:** Aplicações em nuvem/borda.

Pool de Conexões

Um **pool de conexões** é um conjunto pré-estabelecido de conexões de banco de dados mantidas "abertas" e reutilizáveis. Em vez de abrir/fechar uma conexão a cada requisição, as conexões são:

1. Criadas antecipadamente e armazenadas em um "pool";
2. Emprestadas à aplicação quando necessário;
3. Devolvidas ao pool após o uso, sem fechá-las fisicamente.

Mecanismo

1. **Inicialização:**
Ao iniciar, o pool cria N conexões (ex: 10 conexões).
2. **Requisição:**
Quando a aplicação precisa do banco, **empresta** uma conexão livre do pool.
3. **Devolução:**
Após executar a operação, a conexão é **devolvida** ao pool (não fechada).
4. **Escalabilidade:**
Se todas as conexões estiverem ocupadas, novas podem ser criadas (até um limite máximo definido).

Vantagens x Desvantagens

- **Vantagens**
 - **Redução de latência:** Evita overhead de abrir conexões repetidas
 - **Controle de recursos:** Evita esgotar `max_connections` do PostgreSQL
 - **Estabilidade:** Protege contra picos de tráfego
- **Desvantagens**
 - **Complexidade:** Configuração adicional (tamanho do pool, timeout)
 - **Estado sujo:** Conexões podem manter transações ou variáveis de sessão mal gerenciadas
 - **Conexões ociosas:** Consumem memória mesmo inativas

Quando Usar

Cenário	Exemplo	Benefício do Pool
Alta concorrência	Aplicações web com usuários simultâneos	Evita sobrecarga de abrir/fechar conexões repetidamente
Conexões custosas	Bancos em nuvem (latência alta)	Elimina o tempo de handshake (até 20-30ms por conexão)
Microserviços	Funções AWS Lambda ou containers	Evita "cold starts" ao reutilizar conexões existentes
Limitação de recursos	PostgreSQL com <code>max_connections</code> restrito	Controla o consumo total de conexões ativas

Quando NÃO Usar

1. **Aplicações de baixo tráfego:** O overhead de gerenciar o pool pode não compensar.
2. **Operações de longa duração:** Conexões emprestadas por muito tempo esgotam o pool.
3. **Ambientes com transações complexas:** Requer cuidados extras com estados de sessão (ex: variáveis temporárias).

Implementação

Em Aplicações (Ex: Python com Psycopg2):

```
from psycopg2 import pool

# Cria um pool com 5 conexões mínimas e 20 máximas
connection_pool = pool.SimpleConnectionPool(
    minconn=5,
    maxconn=20,
    host="meu.postgres.com",
    database="app_db",
    user="admin",
    password="senha"
)

# Uso:
conn = connection_pool.getconn()
cursor = conn.cursor()
cursor.execute("SELECT * FROM users")
connection_pool.putconn(conn) # Devolve ao pool
```

Via Poolers Externos (Recomendado para alta escala):

- **PgBouncer:**
Leve, ideal para concorrência massiva (modos: session, transaction, statement).
Exemplo de configuração:

```
[databases]
app_db = host=postgres-server dbname=prod user=pooler

[pgbouncer]
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 50
```

- **Pgpool-II:**
Oferece além de pooling: load balancing, failover, replicação.

Melhores Práticas

1. **Defina limites realistas:**
 - $pool_size \approx (n\acute{u}m_workers * 1.5)$
 - Monitore conexões ociosas (SHOW POOLS; no PgBouncer).
2. **Use modo "transaction"** em poolers:
Conexões são devolvidas após cada transação (não após cada query).
3. **Sempre devolva conexões:**
Vazamentos esgotam o pool rapidamente (use blocos try/finally).
4. **Teste sob carga:**
Simule picos para ajustar minconn/maxconn.